

ProofFix

Evidence-closed Kubernetes incident recovery

The winning problem definition, architecture, benchmark, infrastructure plan, and 72-hour execution roadmap for the micro1 Frontier Engineering Challenge 2026.

DON'T TRUST THE DIAGNOSIS. TRUST THE PROOF OF RECOVERY.

15	90	1
paired benchmark cases	total benchmark runs	verified state-transition metric

Prepared for a team-size-one online challenge. Detailed rules were taken from the supplied 10-page micro1 rulebook; live event metadata was checked on HackerEarth.

Executive decision

Build **ProofFix**: an operational agent that turns a Kubernetes alert into a human-approvable Incident Recovery Packet, applies the smallest admissible recovery inside a disposable cluster, and proves that the whole service recovered.

The exact user bottleneck is not merely finding a plausible root cause. It is establishing the causal evidence chain, choosing the correct operation and target, bounding blast radius, preserving rollback, and verifying the post-change state under incident pressure.

Central product promise: “Give me the smallest safe recovery I can approve now, show the evidence that justifies it, and prove the service is healthy afterward.”

Why this is the strongest rubric play

Candidate	User	Agent	E2E	Eval	Repro	Insight	Total
ProofFix: verified Kubernetes recovery	15	28	19	15	14	5	96
CI/test-failure repair	13	23	18	15	15	3	87
Kubernetes compliance evidence	14	25	17	13	12	4	85
FinOps anomaly remediation	14	24	16	12	11	4	81

Rubric-ceiling estimates, not project results. ProofFix leads on differentiation, live end-to-end verification, safety narrative, public scenarios, and a failure-derived hot take. CI repair is easier but crowded; compliance and FinOps have weaker live recovery oracles for this 72-hour window.

The research signal

Evidence	Consequence for the build
ITBench reports 13.8% SRE resolution.	There is substantial measurable headroom on realistic IT tasks.
Cloud-OpsBench reaches 0.76/0.68 joint RCA but only 0.38/0.15 evidence closure.	Correct final answers can still be untrustworthy operationally.
1.675-run RCA failure study finds incomplete exploration and hallucinated telemetry across models.	A stronger prompt is not the solution; workflow controls are.
R2Act finds 91.4%-99.7% root-service accuracy but 36.8%-60.3% valid recovery actions.	Diagnosis and valid action selection are different tasks.
21 Aug trajectory study shows success depends on evidence-grounded paths.	Trajectories and evidence gates belong in the primary design.

Target user, trigger, and output

User. The on-call SRE or platform engineer at a small or mid-sized organization operating Kubernetes microservices, often responding to a service they did not author.

Trigger. An availability or latency alert with distributed evidence across Kubernetes state, logs, metrics, traces, deployment configuration, and service dependencies.

Bottleneck. Under time pressure, the responder must judge whether the diagnosis is supported, whether the proposed action is admissible, what else could be harmed, how to roll back, and whether recovery truly occurred.

Incident Recovery Packet	What the responder receives
Situation	Impact, affected SLO, scope, current health, severity
Proof	Timestamped evidence, dependency/fault path, source pointers
Reasoning	Ranked hypotheses, discriminating tests, counterevidence
Recovery	Smallest action, exact target, expected postcondition
Safety	Blast radius, forbidden actions checked, rollback, approval gate
Verification	Before/after probes, whole-system health, residual uncertainty

Baseline fairness contract

- One general-purpose ReAct agent with the official mitigation task and basic instructions.
- Same fixed model, typed AIOpsLab tools, 20 environment actions, total token ceiling, timeout, fresh cluster, and no evaluation-time internet as ProofFix.
- No explicit evidence ledger, hypothesis board, challenger, safety policy, runbook memory, or post-action verification state machine.
- Every run is reported; no best-of-N selection. Resource differences and internal model-call counts are disclosed.

ProofFix architecture

Stage	Responsibility	Gate / failure behavior
1. Controller	Freeze manifest; enforce budget; trace state	Deterministic state machine
2. Observer	Topology, symptoms, changes, telemetry	Read-only, source every observation
3. Hypothesis manager	Rank causes and next discriminating test	Support + counterevidence required
4. Challenger	Try to falsify leading diagnosis	Return to observation if evidence gaps remain
5. Recovery planner	Smallest reversible action and rollback	Exact operation, target, expected postcondition
6. Safety gate	Allowlist, scope, data-loss and rollback policy	Human approval in production; sandbox in benchmark
7. Executor	Apply approved plan and record diff	One bounded re-plan; no silent retries
8. Health verifier	Whole-system probes and persistence checks	Rollback on regression or partial recovery
9. Report compiler	Build packet from trace only	Unsupported critical claim fails the case

Fifteen-case benchmark

Use all 14 current mitigation tasks in the MIT-licensed [AIOpsLab](#) registry plus one pre-registered healthy/noisy abstention control. AIOpsLab evaluates the general state of the entire system after mitigation, not only the injected resource.

#	Case	Family	Critical capability
1	Target-port: user service	Routing	YAML + exact target
2	Target-port: text service	Routing	Topology transfer
3	Target-port: post-storage	Routing	Persistence dependency
4	Missing MongoDB auth	Auth/config	No secret leakage
5	Revoked auth: geo	Authorization	Backend propagation
6	Revoked auth: rate	Authorization	Target transfer
7	Unregistered user: geo	Identity	Absent vs revoked
8	Unregistered user: rate	Identity	Exact target
9	Hotel app misconfiguration	App config	Cross-layer fix
10	Deployment scaled to zero	Capacity	Minimal restoration
11	Nonexistent node assignment	Scheduling	Counterfactual evidence
12	Kafka queue failure	Messaging	Multi-service path
13	Redeploy without PV	Data safety	Challenge: prevent data-loss masking
14	Wrong binary usage	Runtime	Process/image evidence
15	Healthy + noisy distractors	Abstention	Control: do nothing safely

Primary metric: Verified Recovery Success

A case scores 1 only when the agent submits within budget, the whole-environment evaluator confirms recovery (or unchanged health for case 15), no forbidden action occurred, post-recovery probes pass for the fixed window, and the packet contains no unsupported critical claim. **VRS is the mean across all case-trial pairs.**

15 x 3 x 2	20	0
cases x trials x systems = 90 runs	environment actions per run	allowed cherry-picked runs

- Paired clean environments from one pinned image; alternate baseline/full order by case and trial.
- Fixed model version and temperature 0 where supported; equal total token ceiling and no internet.
- Pre-register cases, acceptable/forbidden actions, metric code, seeds, timeouts, and exclusions.
- Report percentage-point change, paired bootstrap 95% CI, and exact McNemar test.
- Ship fast deterministic replay fixtures plus a full live mode; judges can replay three representative live incidents.

Infrastructure and reproducibility

Resource	Owned role	Fairness / safety constraint
VM2 control plane	Repo, orchestration, trace ingest, dashboard, aggregation	No scenario workload competes with control services
Runner VM A	Baseline shards	Same immutable image as full system
Runner VM B	ProofFix shards	Pair and alternate with A
Runner VM C	Extra trials and live demo cluster	Disposable namespaces/clusters only
Clean-room VM	Zero-cache setup, reproduction, video	No dev credentials or warmed state
Gemini Deep Research	Failure taxonomy, runbook sources, disconfirmation	Freeze before eval; no gold-label access
Antigravity pipelines	Deploy, inject, schedule, trace, infra-retry	Never retry a valid agent failure
Frontier orchestration	Development ablations	Same final model/token ceiling for comparison

Trajectory contract

Every run emits JSONL and a human-readable view containing: instructions; observations; tool calls and responses; hypothesis updates; verifier feedback; policy decision; approval checkpoint; action diff; health checks; retries; final packet; model/tool versions; timestamps; token/cost data; and content hashes. Secrets are redacted at the tool boundary.

Target gates (goals, not current results)

>=80%	+30 pp	0	>=85%	<10 min
VRS	vs baseline	forbidden actions	evidence gates	median TTM

Clean-environment contract

- Pinned OS/container digests, model identifier, dependency lock, scenario commit, and deterministic policy manifest.
- One command for setup, baseline, ProofFix, evaluation, and report generation; expected output and runtime documented.
- Quick replay target under 15 minutes; full 90-run benchmark target under two hours on four runners.
- No credentials in submission; environment variables documented; all public/synthetic data and licenses enumerated.

72-hour execution roadmap

Window	Outcome	Non-negotiable exit gate
Aug 28 - foundation	Freeze problem, cases, metric, fairness, licenses. Scaffold trace schema, baseline, one smoke case.	Baseline completes one live mitigation and records a replayable trace.
Aug 29 - full path	Observer, hypothesis ledger, challenger, policy, executor, verifier, packet. Five dev cases.	One incident runs alert-to-verified-packet without manual repair.
Aug 30 - evidence	Ablations; remove weak components; freeze code; run 90 paired evaluations; aggregate stats.	All runs accounted for; metric report and changelog generated.
Aug 31 - delivery	Clean-room run, README, video, trajectory viewer, licenses, final package.	Freeze 15:00 UTC; verify authenticated countdown before upload.

Improvement changelog plan

Stage	Hypothesis	Keep only if
Baseline	Generic ReAct handles simple incidents	Fair starting point is reproducible
+ Evidence ledger	Prevents premature diagnosis	VRS/grounding rise without excessive latency
+ Challenger	Catches plausible wrong causes	Unsupported claims and invalid actions fall
+ Safety policy	Prevents target/operation mistakes	Forbidden-action rate reaches zero
+ Health verifier	Converts fixes into proved recovery	VRS rises; partial fixes are caught
+ Failure memory	Improves transfer to unseen targets	Held-out VRS rises without leakage/cost blowout

Preserve at least one removed experiment. Prime candidates are unconstrained multi-agent debate (latency/cost) and unfiltered trajectory memory (anchoring/leakage).

Hot take

Root-cause accuracy is a vanity metric when the user needs a safe recovery. The meaningful unit of agent reliability is a verified state transition: evidence closed, action admissible, blast radius bounded, recovery observed, and rollback available. Better prompts may improve prose; they do not supply the missing control system.

Risks, limits, and decision guardrails

Risk	Mitigation
"Another SRE agent"	Lead with evidence closure, policy-gated recovery, and verified state transition—not chat-based RCA.
Live benchmark noise	Pinned images, paired order, three trials, infra-failure classification, deterministic replay fixtures.
Benchmark leakage	Development/evaluation split, no internet, freeze manifests, exclude gold labels and eval traces from memory.
Consequential actions	Disposable clusters, typed allowlist, namespace scope, data-loss policy, rollback, production human approval.
License ambiguity	Redistribute only verified compatible assets. AIOpsLab is MIT; Cloud-OpsBench is research evidence unless its repo license is confirmed.
Time pressure	Baseline and one complete live path first; UI polish only after evaluator and trace are stable.

Material limitations

- AIOpsLab is a benchmark; its faults and service topologies do not represent every production incident.
- Fifteen cases support a strong hackathon comparison, not a universal reliability claim.
- Recognizable scenario patterns may still encourage overfitting; target variants and gold-label isolation reduce but do not eliminate it.
- Human review time is reported only if a qualified reviewer can score blinded packets consistently.
- HackerEarth's deeper dynamic tabs were blocked from this VM by its anonymous/VPN IP gate; the supplied 10-page rulebook controls detailed requirements.

Sources and provenance

Source	Publisher / date	Supports
Agentic Workflows Hackathon (supplied 10-page PDF)	micro1 / 27 Aug 2026	Rubric, rules, deliverables, 10+ cases, video, trajectories
HackerEarth challenge page	HackerEarth / accessed 28 Aug	Dates, online, team size 1, registrations
Effective Troubleshooting	Google SRE	Evidence, hypothesis testing, controlled changes
AIOpsLab paper	Chen et al. / 12 Jan 2025	Live fault injection, whole-system evaluation
AIOpsLab repository	Microsoft / accessed 28 Aug	MIT license and current mitigation registry
ITBench	Jha et al. / 7 Feb 2025	IT automation benchmark and low SRE resolution
Cloud-OpsBench v2	Wang et al. / 22 Aug 2026	Outcome-versus-evidence gap
Systematic RCA failures	Kim et al. / 10 Feb 2026	1,675 runs; prompt-only limits
R2Act	Qi et al. / 6 Jul 2026	Diagnosis-versus-action validity gap
Beyond Fault Localization	Lu et al. / 21 Aug 2026	3,500 trajectory study

Research stop condition

The decision survived a second-wave challenge on novelty, licensing, benchmark availability, reproducibility, action safety, and 72-hour buildability. Additional searches were returning weaker variants and were unlikely to change the selection before implementation needed to begin.